

Formal Proof of Correctness for `list2(n)`

OLD EXAM QUESTION

Problem Statement

Consider the following pseudocode:

```
list2(n):  
1 A ← array[0..n]  
2 x ← 1; y ← 2; s ← 0  
3 A[0] ← s  
4 for i ← 0 to n - 1 do  
5     s ← s + y - x  
6     A[i + 1] ← s  
7     x ← y; y ← 2 · y  
8 return A
```

We want to prove, using a loop invariant, that given a positive integer n , `list2(n)` returns an array:

$$A = [2^0 - 1, 2^1 - 1, 2^2 - 1, \dots, 2^n - 1].$$

Step 1: Analyze the Pattern

Inside the loop:

$$s \leftarrow s + y - x, \quad x \leftarrow y, \quad y \leftarrow 2 \cdot y.$$

Initial values:

$$x = 1, \quad y = 2, \quad s = 0, \quad A[0] = 0.$$

Example of iterations:

- Initially: $s = 0$

- After first iteration: $s = 0 + 2 - 1 = 1$
- Next: $x = 2, y = 4$
- Next iteration: $s = 1 + 4 - 2 = 3$
- Next: $x = 4, y = 8$
- Next iteration: $s = 3 + 8 - 4 = 7$

Pattern: s becomes $1, 3, 7, 15, \dots$ which is $2^i - 1$.

Step 2: Deriving the Loop Invariant

To find the invariant, observe how each variable changes:

- x starts at 1 and doubles each time:

$$x : 1, 2, 4, 8, \dots \implies x = 2^i.$$

- y starts at 2 and doubles each time:

$$y : 2, 4, 8, 16, \dots \implies y = 2^{i+1}.$$

- s starts at 0 and updates as:

$$s \leftarrow s + y - x.$$

If $s = 2^i - 1$ before iteration, then after update:

$$s' = (2^i - 1) + 2^{i+1} - 2^i = 2^{i+1} - 1.$$

So s follows $s = 2^i - 1$.

Thus, at the start of iteration i :

$$x = 2^i, y = 2^{i+1}, s = 2^i - 1, A[j] = 2^j - 1 \text{ for all } 0 \leq j \leq i.$$

Step 3: Prove the Invariant

Initialization

Before the loop:

$$i = 0, x = 1 = 2^0, y = 2 = 2^1, s = 0 = 2^0 - 1, A[0] = 0.$$

Invariant holds.

Maintenance

Assume invariant holds at start of iteration i . Then:

$$s \leftarrow s + y - x = (2^i - 1) + 2^{i+1} - 2^i = 2^{i+1} - 1.$$

So s becomes $2^{i+1} - 1$. Then:

$$A[i + 1] = s, \ x \leftarrow y = 2^{i+1}, \ y \leftarrow 2 \cdot y = 2^{i+2}.$$

Invariant holds for next iteration.

Termination

After n iterations:

$$s = 2^n - 1, \ A = [2^0 - 1, 2^1 - 1, \dots, 2^n - 1].$$

Step 4: Conclusion

By the loop invariant, `list2(n)` returns:

$$A = [2^0 - 1, 2^1 - 1, 2^2 - 1, \dots, 2^n - 1].$$